

Lab Sheet: Designing and Implementing a Simple General-Purpose Microprocessor

A 4-bit Accumulator-Based CPU in Logisim-Evolution

Computer Organization / Digital Systems Laboratory

1 Overview

In this lab you will design and build a complete, working **general-purpose CPU** inside Logisim-Evolution, starting from a provided Arithmetic Logic Unit (ALU) module (`alu.circ`) and adding the remaining components required to make a programmable machine: a **Program Counter (PC)**, an **Instruction Register (IR)**, an **Accumulator (ACC)**, a **RAM/ROM** program memory, and a small amount of **hardwired control logic**.

This machine deliberately uses the simplest possible organisation: **the PC is the only address ever presented to memory** (there is no separate address register, and no shared bus – every connection is a direct, point-to-point wire), and **there is no fetch/decode/execute state machine to design**. Instead, every instruction's opcode bits, sitting right there in the instruction word coming out of RAM, are wired straight into the handful of gates that generate the load-enable and ALU-select signals. The whole CPU updates its registers once per clock edge – it is a *single-cycle* machine.

The instruction set is deliberately small (**9 instructions, including one conditional and one unconditional branch**) so that the whole machine stays small enough to design, debug, and fully understand within the lab session.

Learning Outcomes

By the end of this lab you will be able to:

- Explain how a CPU's datapath (registers, ALU, point-to-point wiring) executes an instruction in a single clock cycle.
- Derive simple combinational (*hardwired*) control equations directly from an opcode encoding, without building a state machine.
- Wire registers, memory and an ALU together in Logisim-Evolution.
- Encode a small instruction set architecture (ISA) into binary opcodes, choosing the encoding so control logic stays simple.
- Hand-assemble and run simple machine-code programs (including loops and conditional branches) on your own CPU.

2 Provided Component: The ALU (`alu.circ`)

You are given a working 4-bit ALU circuit, `ALU_4`, with the following interface:

Pin	Width	Description
A	4 bits	First operand (input)
B	4 bits	Second operand (input)
S	3 bits	Function-select (input)
F	4 bits	Result (output)

Internally, A and B feed an **Adder**, a **Subtractor**, a 4-bit **AND** gate and a 4-bit **OR** gate in parallel; an 8:1 **Multiplexer** controlled by S selects which result reaches F . Only four of the eight possible select codes are wired up, giving the following function table:

$S_2S_1S_0$	Operation	F
000	ADD	$A + B$
001	SUB	$A - B$
010	AND	$A \wedge B$
011	OR	$A \vee B$
100--111	—	unused / reserved

Task 1 — Familiarisation. Open `alu.circ` in Logisim-Evolution. Test each of the four functions with a few input combinations using the Poke tool, and confirm the function table above by observation. Keep a note of the S codes: you will wire S directly from opcode bits later (§3).

3 CPU Architecture

You will build a small **single-accumulator CPU** with 4-bit data width (matching the ALU) and a program/data memory of **16 locations** $\times$ **8 bits** (4-bit address, 8-bit word). Following the style of a classic hardwired-control datapath (Figure 1):

- **No shared bus.** Every connection is a dedicated point-to-point wire (or, where more than one value can reach the same place, a small 2:1 multiplexer) – never a multi-driver line that needs arbitration.
- **No MAR.** RAM’s address input is fed by a single 2:1 mux choosing directly between PC (fetching an instruction) and $IR[3:0]$ (addressing the operand of the instruction just fetched). A multiplexer is combinational – it is not a register, so there is still no separate “address register” anywhere in the machine.
- **No fetch/decode/execute state machine.** All of the load-enable and select signals ($IRload$, $PCload$, $MemWr$, $ACCload$, ...) are generated by a small block of *combinational* logic that reads the opcode bits sitting in IR directly – this is called **hardwired control**, as opposed to a microprogrammed sequencer with states. The only sequential element besides the datapath’s own registers is a single 1-bit *Fetch/Execute phase* flip-flop, T (Figure 2) – not a multi-state FSM.

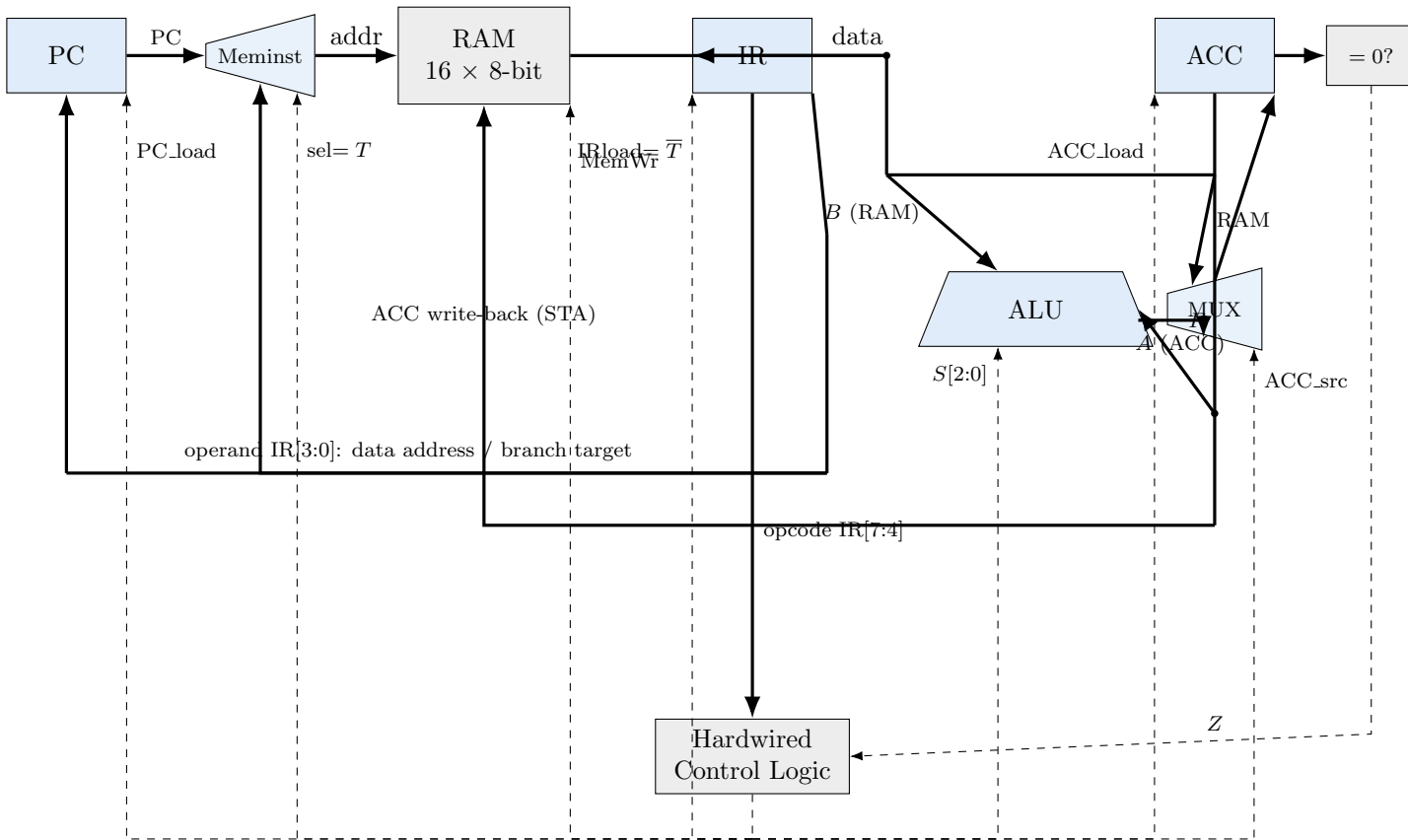


Figure 1: Datapath overview – no shared bus, no MAR. A single **Meminst** mux (not a register) selects whether RAM’s address comes from PC (instruction fetch) or from IR[3:0] (the operand address, for LDA/STA/ADD/SUB/AND/OR and as the branch target for JMP/JZ). RAM’s data output feeds IR (on fetch) and the ALU’s *B* input / ACC-source mux (on execute); ACC feeds the ALU’s *A* input, a zero-flag detector, and RAM’s data-in port (for STA). All load-enables, the mux selects and $S[2:0]$ come from the Hardwired Control Logic, driven purely by IR’s opcode bits, the zero flag *Z*, and the phase bit *T*.

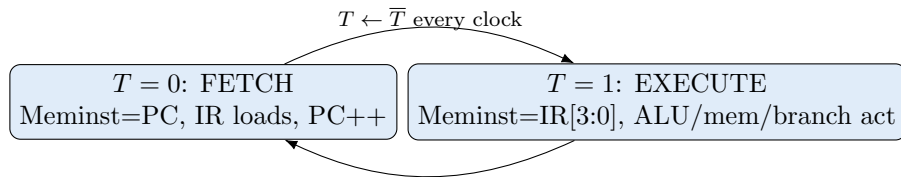


Figure 2: The only sequencing element: a single toggle flip-flop *T* (Fetch/Execute phase), *not* a multi-state FSM. Everything else in §4’s control table is a pure function of the current opcode, *T*, and the zero flag.

Component Summary

Block	Purpose
PC (Program Counter)	4-bit register; increments at the end of FETCH ($T=0 \rightarrow 1$); loaded from IR[3:0] instead, at the end of EXECUTE , when PC_load is asserted (branch taken).
Meminst mux	2:1 multiplexer (not a register) selecting RAM’s address: PC when $T=0$, IR[3:0] when $T=1$.
RAM	16×8 -bit memory holding program and data. Address from the Meminst mux; Din =ACC, WE = MemWr (write only for STA, only at $T=1$).
IR (Instruction Register)	8-bit register, loaded from RAM’s data output only during FETCH ($T=0$). Its opcode nibble feeds the Hardwired Control Logic; its operand nibble feeds the Meminst mux and PC’s branch-load input.

4 Instruction Set Architecture

Each instruction occupies **one 8-bit memory word**: a 4-bit **opcode** in the upper nibble and a 4-bit **operand** (a memory address, doubling as a branch target) in the lower nibble.

Opcode	Mnemonic	Operation	ALU <i>S</i>
0000	NOP	No operation.	—
0001	LDA addr	$ACC \leftarrow M[addr]$	—
0010	STA addr	$M[addr] \leftarrow ACC$	—
0011	ADD addr	$ACC \leftarrow ACC + M[addr]$	000
0100	SUB addr	$ACC \leftarrow ACC - M[addr]$	001
0101	AND addr	$ACC \leftarrow ACC \wedge M[addr]$	010
0110	OR addr	$ACC \leftarrow ACC \vee M[addr]$	011
0111	JMP addr	$PC \leftarrow addr$ (unconditional branch)	—
1000	JZ addr	if $ACC = 0$: $PC \leftarrow addr$ (conditional branch)	—
1001	HLT	Halt: gate off the clock enable.	—

Table 1: The 10-instruction ISA. Opcodes 1010–1111 are unused and reserved.

Hardwired Control Logic

Two of the eight control signals need no decode logic at all – they are a direct (inverted, for one) wire from the phase flip-flop T : $\text{Meminst_sel} = T$ and $\text{IRload} = \overline{T}$. The remaining six depend on the opcode $O_3O_2O_1O_0 = \text{IR}[7:4]$ (and, for PC_load , the zero flag Z), and are only meant to act during **EXECUTE** ($T=1$):

Opcode	Mnemonic	ACC_load	ACC_src	MemWr	PC_load	HLT
0000	NOP	0	—	0	0	0
0001	LDA	1	RAM	0	0	0
0010	STA	0	—	1	0	0
0011	ADD	1	ALU	0	0	0
0100	SUB	1	ALU	0	0	0
0101	AND	1	ALU	0	0	0
0110	OR	1	ALU	0	0	0
0111	JMP	0	—	0	1	0
1000	JZ	0	—	0	Z	0
1001	HLT	0	—	0	0	1

(all six columns are additionally ANDed with T , since they must stay inactive during **FETCH**). ACC_src selects RAM data for LDA and the ALU result for the four arithmetic/logic opcodes; $S[2:0]$ is 000/001/010/011 for ADD/SUB/AND/OR respectively and don't-care otherwise. Every column here is a small, fixed function of 4 (or 5) input bits – buildable directly as a two-level AND–OR circuit, or as a tiny lookup table – with *no* states, and *no* sequencing beyond the single phase bit T . This is what it means for **the control signal to live in the instruction itself**, decoded freshly every cycle, rather than in a separate multi-state control unit.

Task 1.5 — Derive it yourself. Before wiring up the table above, pick one column (e.g. ACC_load) and derive its minimal AND–OR expression directly from the ISA table yourself. Compare your gate count against a classmate's.

5 Build Tasks

Work through the tasks in order. Test each block in isolation before wiring it into the full CPU — debugging a single register is far easier than debugging the whole machine at once.

Task 2 — Registers. Build 4-bit PC and ACC, and an 8-bit IR, using Logisim’s **Register** component, each with a separate *load enable* input. There is **no MAR** to build. Add the 1-bit phase flip-flop T (a D flip-flop wired as $T \leftarrow \overline{T}$, i.e. toggling every clock edge).

Task 3 — Memory and the Meminst mux. Add a 16×8 -bit **RAM** component. Build the **Meminst** 2:1 multiplexer feeding its address input directly from PC (when $T=0$) or IR[3:0] (when $T=1$) – this replaces the MAR entirely. Use the *Load Image* feature (right-click → “Load Image”) to pre-load a hex program image once you have written one (§6).

Task 4 — ALU and ACC-source mux. Wire ACC to the ALU’s A input, and RAM’s data output (tapped *before* IR) to the ALU’s B input, as point-to-point wires – no bus. Build the ACC-source mux (RAM data vs. ALU result F) feeding ACC’s data input, and wire ACC’s output to RAM’s data-in port for STA.

Task 5 — Hardwired Control Logic. Build the six opcode-decoded signals from the truth table in §4 (ACC_load, ACC_src, MemWr, PC_load, $S[2:0]$, HLT) as plain combinational logic (AND/OR/NOT gates, or a small lookup table) reading IR[7:4] and the zero flag Z . Remember to AND each of them with T so they are inactive during FETCH. This is *not* a state machine – there are no flip-flops in this task at all.

Task 6 — Assemble the CPU. Wire Tasks 2–5 together following Figure 1. Add a manual clock (or Logisim’s clock component) with single-step capability so you can observe FETCH and EXECUTE separately on the Poke tool / Logisim’s built-in *Hex Editor* and *Log*.

6 Test Program

Hand-assemble and run the following program, which computes $5 + 3$, stores the result, then loops decrementing a counter down to zero using the conditional branch. All 16 words of memory (addresses 0–F) are used, with instructions in 0–8 and data/constants in B–F: no address is shared between code and data.

```
; addr : instruction ; encoding (opcode|operand) ; comment
0 : LDA E ; 0001 1110 ; ACC <- M[E] = 5
1 : ADD F ; 0011 1111 ; ACC <- ACC + M[F] = 5+3 = 8
2 : STA D ; 0010 1101 ; M[D] <- ACC (result = 8)
3 : LDA C ; 0001 1100 ; ACC <- M[C] (loop counter)
4 : SUB B ; 0100 1011 ; ACC <- ACC - M[B] (counter - 1)
5 : STA C ; 0010 1100 ; M[C] <- ACC (store new counter)
6 : JZ 8 ; 1000 1000 ; if ACC = 0 goto HLT
7 : JMP 3 ; 0111 0011 ; else repeat loop
8 : HLT ; 1001 0000 ; stop
9 : - ; 0000 0000 ; unused
A : - ; 0000 0000 ; unused
B : 1 (constant) ; 0000 0001 ; decrement amount
C : 3 (variable) ; 0000 0011 ; loop counter, initial value 3
D : 0 (variable) ; 0000 0000 ; result storage
E : 5 (constant) ; 0000 0101 ; first operand
F : 3 (constant) ; 0000 0011 ; second operand
```

Task 7 — Verify. Trace the program by hand first (fill in a register-value table for PC/IR/ACC/M[C] on every clock tick), then run it in Logisim and confirm your trace matches. $\square$ $M[D]$ ends

at 1000 (8). $\square$ $M[C]$ ends at 0000 (0) after three passes through the loop. $\square$ The CPU halts (does not keep fetching after HLT).

Task 8 — Your own program. Write and test a second program that uses AND/OR and at least one branch, of your own design (e.g. masking a value, or a 3-iteration loop that toggles a memory location).

7 Deliverables

- Your completed `.circ` file, containing the ALU (unmodified internals permitted to be re-wrapped as a subcircuit), registers, memory, the Meminst/ACC-source muxes, the Hardwired Control Logic, and the top-level CPU circuit.
- A short design note (1 page) showing the truth table or Boolean equations you actually built for the six opcode-decoded control signals.
- Hand-trace tables and machine-code listings for both test programs (§6 and Task 8).

8 Marking Guide

Criterion	Marks
ALU correctly integrated and verified	10
Registers (PC, IR, ACC) and phase flip-flop T correct, independently testable	15
Meminst mux and RAM correctly addressed (no MAR)	10
Hardwired Control Logic correctly implements the §4 table	30
All 10 instructions function correctly	20
Test program (§6) runs correctly	10
Student-authored program (Task 8)	5
Total	100

Extension (Optional, not marked)

Extend the ISA with an OUT instruction driving a 7-segment display, or widen the datapath to 8 bits. Consider what changes to the Hardwired Control Logic and ALU are required if you add an XOR or NOT function using the two still-unused ALU select codes.